



순위표용 Docker Image 제출 규정: 글쓰기 채점 능력 평가

이 문서는 Docker 이미지를 제출하는 참가자를 위한 지침입니다. 오케스트레이터는 제출된 이미지를 `docker pull` 한 뒤, 아무 추가 인자 없이 그대로 실행하고, 그 컨테이너를 OpenAI 호환 모델 서버로 간주하여 평가 서버에 연결합니다.

이하의 내용은 현재 `orchestrator/server.py`의 구현을 기준으로 실제 동작 방식을 기술합니다.

1. 전체 동작 흐름

Docker 이미지 제출이 들어오면 시스템은 아래 순서로 동작한다.

- 리더보드에서 다음 평가 대기 모델을 조회한다.
- 제출 정보의 `modelUrl`을 Docker 이미지 주소로 해석한다.
- 필요하면 `docker://` prefix를 제거한 뒤 `docker pull <image>`를 수행한다.
- 오케스트레이터가 아래와 같은 형태로 컨테이너를 실행한다.
- `GET /health`로 헬스 체크가 통과할 때까지 대기한다.
- 헬스 체크가 통과하면 `GET /v1/models`를 통해 실제 모델 이름을 읽는다.
- 평가 서버에 `POST /evaluate`를 보내고, 모델 API 주소로 `http://vllm-job-<leaderboard_id>:8000`를 전달한다.
- 평가 서버는 해당 컨테이너의 `POST /v1/chat/completions`를 호출하여 벤치마크를 수행한다.
- 평가가 완료되면 평가 서버가 오케스트레이터의 `/webhook/eval-complete`를 호출한다.
- 오케스트레이터는 제출 컨테이너를 종료하고, Docker 제출인 경우 이미지까지 `docker rmi -f`로 정리한다.

즉, 제출 이미지는 “평가 시점에 독립 실행 가능한 모델 API 서버”여야 한다.

2. 오케스트레이터가 실제로 실행하는 방식

Docker 이미지를 제출한 경우 실행되는 명령 흐름은 개념적으로 아래와 같다.

```
docker pull <YOUR_IMAGE>

docker run -d \
  --name vllm-job-<leaderboard_id> \
  --network eval-net \
  --gpus all \
  <YOUR_IMAGE>
```

중요한 점은 다음과 같다.

- 오케스트레이터는 이미지 뒤에 추가 커맨드 인자를 붙이지 않는다.
- `p 8000:8000` 같은 포트 publish 를 하지 않는다.
- 평가 서버는 호스트 포트가 아니라 Docker 네트워크 `eval-net` 내부에서 컨테이너 이름으로 접근한다.
- 따라서 컨테이너 내부 서버는 반드시 `0.0.0.0:8000`에 bind 되어 있어야 한다.
- 서버가 `127.0.0.1:8000`에만 떠 있으면 다른 컨테이너에서 접근할 수 없다.

즉, 참가자가 만든 이미지는 `docker run <image>`만으로 바로 서버가 시작되어야 한다.

3. 제출 이미지가 반드시 만족해야 하는 계약

3.1. 네트워크/프로세스 계약

- 컨테이너 시작만으로 모델 서버가 자동 실행되어야 한다.
- 메인 프로세스가 **foreground**에서 살아 있어야 한다.
- 서버는 **0.0.0.0:8000** 에서 HTTP 요청을 받아야 한다.
- GPU 환경에서 동작해야 하며, 실행 시 **-gpu** all 이 전달된다.

3.2. 필수 엔드포인트

다음 엔드포인트는 사실상 필수이다.

- **GET /health**
- **GET /v1/models**
- **POST /v1/chat/completions**

현재 코드상 **/health** 는 헬스 체크에 직접 사용되고, **/v1/models** 는 평가 요청 시 모델 이름을 확인하는 데 사용된다. **/v1/models** 가 완전히 없으면 fallback 경로가 있긴 하지만, 정확한 모델 식별과 안정성을 위해 반드시 구현하는 것을 권장하는 수준이 아니라 거의 필수라고 보는 것이 맞다.

4. 각 API에서 기대하는 최소 동작

GET /health

- 용도: 컨테이너 준비 완료 확인
- 기대값: **200 OK**
- 권장 응답 예시: **{ "status": "ok" }**

모델 로딩이 오래 걸린다면, 준비되기 전까지는 503 등을 반환해도 되지만 최종적으로는 200으로 바뀌어야 한다. 오케스트레이터는 최대 30분 동안 5초 간격으로 재시도한다.

GET /v1/models

- 용도: 실제 서빙 모델의 식별자 확인
- 기대값: Open AI 스타일 모델 목록 JSON
- 최소 예시:

```
{
  "object": "list",
  "data": [
    {
      "id": "my-model",
      "object": "model",
      "owned_by": "submission"
    }
  ]
}
```

오케스트레이터는 **data[0].id** 를 읽어서 평가 서버에 전달할 **model_name**으로 사용한다.



POST /v1/chat/completions

- 용도: 실제 벤치마크 추론 호출
- 요청 형식: OpenAI Chat Completions 호환
- 최소 요청 필드: `model`, `messages`, `max_tokens`, `temperature`, `top_p`, `seed`(가능하면 지원), `stop`(가능하면 지원)
- 최소 응답 예시:

```
{
  "id": "chatcmpl-001",
  "object": "chat.completion",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "답변 텍스트"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 20,
    "total_tokens": 30
  }
}
```

`usage` 는 가능하면 제공하는 것이 좋지만, 시스템마다 일부 필드는 생략될 수도 있다. 다만 `choices[0].message.content` 는 정상적으로 들어 있어야 한다.

5. 평가 서버가 실제로 전달하는 값

오케스트레이터는 평가 서버에 대략 아래 형태의 설정을 전달한다.

```
{
  "model_name": "<resolved from /v1/models>",
  "vllm_backend_config": {
    "model_api": "http://vllm-job-<leaderboard_id>:8000",
    "api_endpoint_path": "/v1/chat/completions",
    "max_tokens": 2048,
    "temperature": 0.0,
    "top_p": 1.0,
  }
}
```



```
"seed": 42,
"stop": ["Q:", "User:"]
}
}
```

즉, 제출 이미지는 OpenAI Chat Completions 스타일 요청을 받을 준비가 되어 있어야 한다. 또한 중요한 점은 다음과 같다.

- 요청의 **model** 필드는 **/v1/models** 에서 노출한 이름과 일치하는 편이 가장 안전하다.
- 컨테이너는 한 번 뜯 뒤 여러 추론 요청을 순차 또는 소수 병렬로 처리할 수 있어야 한다.
- 오케스트레이터는 Docker 제출 이미지에 대해 **-max-model-len**, **-tensor-parallel-size** 같은 vLLM 인자를 대신 넣어주지 않는다.
- vLLM 외 다른 서빙 엔진이나 직접 구현한 서버를 쓴다면, 긴 입력도 처리할 수 있도록 최대 입력 길이와 관련 설정을 이미지 내부에서 직접 보장해야 한다.

즉, 이런 값들은 참가자 이미지 안에서 직접 결정해야 한다.

6. 어떤 이미지를 만들어야 하는가

가장 안전한 방식은 아래 둘 중 하나이며 대부분의 경우 1번이 가장 쉽다.

- vLLM 기반 이미지를 만들고, entrypoint 에서 고정된 모델을 **vllm serve** 로 실행
- 직접 FastAPI/Flask 등으로 OpenAI 호환 래퍼 서버를 구현

권장 구조

- 이미지 안에 모델 서빙 코드가 포함되어 있어야 한다.
- **CMD** 또는 **ENTRYPOINT** 만으로 서버가 떠야 한다.
- 별도 수동 입력이나 추가 인자 없이 실행되어야 한다.
- 가능하면 이미지 태그 하나가 하나의 고정 모델/설정을 의미하도록 만드는 것이 좋다.

7. vLLM 기반 예시

아래 예시는 고정 모델을 자동으로 띄우는 가장 단순한 패턴이다.

예시 **Dockerfile**

```
FROM vllm/vllm-openai:latest

ENV MODEL_NAME=Qwen/Qwen2.5-7B-Instruct
ENV SERVED_MODEL_NAME=qwen2.5-7b-instruct-submission
ENV MAX_MODEL_LEN=32768
ENV TENSOR_PARALLEL_SIZE=1
ENV PIPELINE_PARALLEL_SIZE=1
ENV DTYPE=bfloat16

COPY entrypoint.sh /entrypoint.sh
```



```
RUN chmod +x /entrypoint.sh

EXPOSE 8000

ENTRYPOINT ["/entrypoint.sh"]
```

예시 `entrypoint.sh`

```
#!/usr/bin/env bash
set -euo pipefail
exec vllm serve "${MODEL_NAME}" \
--host 0.0.0.0 \
--port 8000 \
--served-model-name "${SERVED_MODEL_NAME}" \
--tensor-parallel-size "${TENSOR_PARALLEL_SIZE}" \
--pipeline-parallel-size "${PIPELINE_PARALLEL_SIZE}" \
--dtype "${DTYPE}" \
--max-model-len "${MAX_MODEL_LEN}"
```

이 방식이면 `/health` , `/v1/models` , `/v1/chat/completions` 가 vLLM 표준 구현으로 함께 제공된다.

8. 직접 API 서버를 구현하는 경우

직접 서빙 서버를 만들 수도 있다. 이 경우에도 반드시 아래를 만족해야 한다.

- `0.0.0.0:8000` 에서 listen
- `GET /health` 지원
- `GET /v1/models` 지원
- `POST /v1/chat/completions` 지원
- OpenAI 스타일 `messages` 입력을 받아 텍스트 응답 생성
- 긴 입력이 들어와도 처리 가능하도록 컨텍스트 길이, 전처리, 메모리 사용 설정을 참가자 쪽에서 직접 맞춰 둘 것

직접 구현은 자유도가 높지만, OpenAI 호환성 차이로 평가 중 예기치 않은 실패가 날 가능성이 있으므로 특별한 이유가 없으면 vLLM 기반이 더 안전하다.

9. 이미지 빌드 및 업로드

오케스트레이터는 `modelUrl` 값을 `docker pull` 가능한 이미지 주소로 사용한다. 예를 들어 아래처럼 빌드하고 푸시할 수 있다.

```
docker build -t my-dockerhub-username/my-model:v1.0.0 .
docker push my-dockerhub-username/my-model:v1.0.0
```

리더보드에 제출할 `modelUrl` 예시는 아래와 같다.

- `docker://my-dockerhub-username/my-model:v1.0.0`
- `my-dockerhub-username/my-model:v1.0.0`



- registry.example.com/team/my-model:2026-03-31

현재 오케스트레이터는 docker:// prefix 가 있으면 제거하고 사용한다.

권장 사항:

- mutable tag 대신 고정 버전 태그 사용
- 평가 환경에서 접근 가능한 공개 또는 사전 인증된 레지스트리 사용
- 첫 실행 시 외부 다운로드가 너무 오래 걸리지 않도록 주의

10. 로컬에서 제출 형태 그대로 검증하는 방법

실제 평가와 가장 비슷하게 확인하려면 로컬에서도 추가 인자 없이 띄워야 한다.

```
docker run --rm --gpus all -p 8000:8000 <YOUR_IMAGE>
```

그 다음 아래를 점검한다.

```
curl <http://localhost:8000/health>
curl <http://localhost:8000/v1/models>
```

예시 추론 요청:

```
curl <http://localhost:8000/v1/chat/completions> \
  -H "Content-Type: application/json" \
  -d '{
    "model": "my-model",
    "messages": [
      {"role": "user", "content": "안녕하세요. 한 줄로 자기소개해
주세요."}
    ],
    "max_tokens": 64,
    "temperature": 0.0,
    "top_p": 1.0,
    "seed": 42
  }'
```

여기서 아래가 모두 만족되면 제출 가능성이 높다.

- 컨테이너가 추가 인자 없이 바로 뜬다.
- /health 가 정상 응답한다.
- /v1/models 가 유효한 id 를 반환한다.
- /v1/chat/completions 가 OpenAI 형식 응답을 반환한다.

11. 자주 발생하는 실패 원인

- 서버가 127.0.0.1:8000 에만 bind 되어 다른 컨테이너에서 접근 불가
- POST /v1/chat/completions 는 있는데 GET /health 또는 GET /v1/models 가 없음
- 컨테이너 시작만으로 모델이 안 뜨고, 추가 커맨드나 환경변수를 따로 넣어야만 동작함



- 이미지가 너무 커서 `docker pull` 시간이 과도하게 길어짐
- 시작 시 모델 다운로드가 오래 걸려 헬스 체크 제한 시간 안에 준비되지 않음
- 요청/응답 형식이 OpenAI Chat Completions 와 달라 평가 서버가 파싱 실패
- 직접 구현한 서버가 짧은 입력만 가정하고 있어 긴 프롬프트에서 길이 초과, 토큰나이지 오류, OOM 등으로 실패

12. 제출 전 체크리스트

- `docker run <image>` 만으로 자동 서버가 시작되는가
- 서버가 `0.0.0.0:8000` 에서 listen 하는가
- `GET /health` 가 200 을 반환하는가
- `GET /v1/models` 가 `data[0].id` 를 포함해 반환되는가
- `POST /v1/chat/completions` 가 OpenAI 호환 응답을 반환하는가
- 긴 입력 요청에도 길이 제한 오류나 비정상 종료 없이 응답하는가
- GPU 환경에서 정상 구동되는가
- 레지스트리에서 `docker pull` 가능한 주소를 제출했는가
- 동일 태그가 나중에 바뀌지 않도록 고정 버전을 사용했는가

13. 한 줄 요약

이 시스템의 Docker 제출은 "이미지를 받아서 `eval-net` 에 바로 띄우고, 그 컨테이너를 `:8000` OpenAI 호환 모델 API 서버로 사용한다"는 계약이다. 따라서 제출 이미지는 반드시 다음 조건을 만족해야 한다.

- `docker run <image>` 만으로 실행 가능
- `0.0.0.0:8000` 에서 서버
- `GET /health` , `GET /v1/models` , `POST /v1/chat/completions` 지원
- GPU 환경에서 안정적으로 응답